# Data Structures & Algorithms

## Exercise 1: Inventory Management System

### Program :

```
import java.util.HashMap;

class Product {
    int productId;
    String productName;
    int quantity;
    double price;

    Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }

    public String toString() {
        return productId + " " + productName + " Qty:" + quantity + " Price:" + price;
    }
}

public class Main {
    static HashMap<Integer, Product> inventory = new HashMap<>();

    static void addProduct(Product p) {
        inventory.put(p.productId, p);
    }

    static void updateProduct(int id, int quantity) {
        if (inventory.containsKey(id))
            inventory.get(id).quantity = quantity;
    }

    static void deleteProduct(int id) {
        inventory.remove(id);
    }

    static void displayProducts() {
        for (Product p : inventory.values())
            System.out.println(p);
    }

    public static void main(String[] args) {

        addProduct(new Product(101, "Laptop", 10, 50000));
        addProduct(new Product(102, "Mouse", 50, 500));

        System.out.println("Inventory:");
        displayProducts();

        updateProduct(101, 15);
        deleteProduct(102);
```

```
        System.out.println("\nAfter Update/Delete:");
        displayProducts();
    }
}
```

## Output :

```
Inventory:
101 Laptop Qty:10 Price:50000.0
102 Mouse Qty:50 Price:500.0

After Update/Delete:
101 Laptop Qty:15 Price:50000.0
```

---

## Exercise 2: E-commerce Platform Search Function

## Program :

```
class Product {
    int productId;
    String productName;
    String category;

    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
}

public class Main {

    static int linearSearch(Product[] products, int id) {
        for (int i = 0; i < products.length; i++) {
            if (products[i].productId == id)
                return i;
        }
        return -1;
    }

    static int binarySearch(Product[] products, int id) {
        int low = 0, high = products.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (products[mid].productId == id)
                return mid;
            else if (products[mid].productId < id)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return -1;
    }
```

```
    public static void main(String[] args) {

        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Mouse", "Electronics"),
            new Product(103, "Keyboard", "Electronics"),
            new Product(104, "Monitor", "Electronics")
        };

        int id = 103;

        System.out.println("Linear Search Index: "
                + linearSearch(products, id));

        System.out.println("Binary Search Index: "
                + binarySearch(products, id));
    }
}
```

## Output :

```
Linear Search Index: 2
Binary Search Index: 2
```

---

## Exercise 3: Sorting Customer Orders

## Program :

```
class Order {
    int orderId;
    String customerName;
    double totalPrice;

    Order(int orderId, String customerName, double totalPrice) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }
}

public class Main {

    static void bubbleSort(Order[] orders) {
        for (int i = 0; i < orders.length - 1; i++) {
            for (int j = 0; j < orders.length - i - 1; j++) {
                if (orders[j].totalPrice > orders[j + 1].totalPrice) {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }

    static int partition(Order[] orders, int low, int high) {
        double pivot = orders[high].totalPrice;
        int i = low - 1;
```

```
        for (int j = low; j < high; j++) {
            if (orders[j].totalPrice < pivot) {
                i++;
                Order temp = orders[i];
                orders[i] = orders[j];
                orders[j] = temp;
            }
        }

        Order temp = orders[i + 1];
        orders[i + 1] = orders[high];
        orders[high] = temp;

        return i + 1;
    }

    static void quickSort(Order[] orders, int low, int high) {
        if (low < high) {
            int pi = partition(orders, low, high);
            quickSort(orders, low, pi - 1);
            quickSort(orders, pi + 1, high);
        }
    }

    static void display(Order[] orders) {
        for (Order o : orders)
            System.out.println(o.orderId + " " + o.customerName + " " + o.totalPrice);
    }

    public static void main(String[] args) {

        Order[] orders = {
            new Order(1, "Bharath", 5000),
            new Order(2, "Rahul", 2000),
            new Order(3, "Kiran", 8000)
        };

        quickSort(orders, 0, orders.length - 1);

        System.out.println("Sorted Orders:");
        display(orders);
    }
}
```

## Output :

```
Sorted Orders:
2 Rahul 2000.0
1 Bharath 5000.0
3 Kiran 8000.0
```

---

## Exercise 4: Employee Management System

## Program :

```
class Employee {
```

```
    int employeeId;
    String name;
    String position;
    double salary;

    Employee(int employeeId, String name, String position, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }
}

public class Main {

    static Employee[] employees = new Employee[10];
    static int count = 0;

    static void addEmployee(Employee e) {
        employees[count++] = e;
    }

    static void searchEmployee(int id) {
        for (int i = 0; i < count; i++) {
            if (employees[i].employeeId == id) {
                System.out.println("Found: " + employees[i].name);
                return;
            }
        }
        System.out.println("Employee Not Found");
    }

    static void traverse() {
        for (int i = 0; i < count; i++) {
            System.out.println(employees[i].employeeId + " " +
                               employees[i].name);
        }
    }

    static void deleteEmployee(int id) {
        for (int i = 0; i < count; i++) {
            if (employees[i].employeeId == id) {
                for (int j = i; j < count - 1; j++)
                    employees[j] = employees[j + 1];
                count--;
                break;
            }
        }
    }

    public static void main(String[] args) {

        addEmployee(new Employee(101, "Bharath", "Developer", 50000));
        addEmployee(new Employee(102, "Rahul", "Tester", 40000));

        traverse();

        searchEmployee(101);

        deleteEmployee(102);
```

```
        System.out.println("After Deletion:");
        traverse();
    }
}
```

## Output :

```
101 Bharath
102 Rahul
Found: Bharath
After Deletion:
101 Bharath
```

---

## Exercise 5: Task Management System

## Program :

```
class Task {
    int taskId;
    String taskName;
    String status;
    Task next;

    Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
    }
}

public class Main {

    static Task head = null;

    static void addTask(int id, String name, String status) {
        Task newTask = new Task(id, name, status);

        if (head == null) {
            head = newTask;
            return;
        }

        Task temp = head;
        while (temp.next != null)
            temp = temp.next;

        temp.next = newTask;
    }

    static void searchTask(int id) {
        Task temp = head;

        while (temp != null) {
            if (temp.taskId == id) {
                System.out.println("Found: " + temp.taskName);
                return;
```

```
            }
            temp = temp.next;
        }

        System.out.println("Task Not Found");
    }

    static void traverse() {
        Task temp = head;

        while (temp != null) {
            System.out.println(temp.taskId + " " + temp.taskName);
            temp = temp.next;
        }
    }

    static void deleteTask(int id) {
        if (head == null) return;

        if (head.taskId == id) {
            head = head.next;
            return;
        }

        Task temp = head;

        while (temp.next != null && temp.next.taskId != id)
            temp = temp.next;

        if (temp.next != null)
            temp.next = temp.next.next;
    }

    public static void main(String[] args) {

        addTask(1, "Design", "Pending");
        addTask(2, "Coding", "In Progress");

        traverse();

        searchTask(2);

        deleteTask(1);

        System.out.println("After Deletion:");
        traverse();
    }
}
```

## Output :

```
1 Design
2 Coding
Found: Coding
After Deletion:
2 Coding
```

---

## Exercise 6: Library Management System

## Program :

```
class Book {
    int bookId;
    String title;
    String author;

    Book(int bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }
}

public class Main {

    static int linearSearch(Book[] books, String key) {
        for (int i = 0; i < books.length; i++) {
            if (books[i].title.equalsIgnoreCase(key))
                return i;
        }
        return -1;
    }

    static int binarySearch(Book[] books, String key) {
        int low = 0, high = books.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
            int cmp = books[mid].title.compareToIgnoreCase(key);

            if (cmp == 0)
                return mid;
            else if (cmp < 0)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return -1;
    }

    public static void main(String[] args) {

        Book[] books = {
            new Book(1, "C", "Author1"),
            new Book(2, "Java", "Author2"),
            new Book(3, "Python", "Author3")
        };

        System.out.println("Linear Search: "
                + linearSearch(books, "Java"));

        System.out.println("Binary Search: "
                + binarySearch(books, "Java"));
    }
}
```

## Output :

Linear Search: 1
Binary Search: 1

---

## Exercise 7: Financial Forecasting

## Program :

```
public class Main {

    static double futureValue(double amount, double rate, int years) {
        if (years == 0)
            return amount;

        return futureValue(amount * (1 + rate), rate, years - 1);
    }

    public static void main(String[] args) {

        double presentValue = 1000;
        double growthRate = 0.10; // 10%
        int years = 3;

        System.out.println("Future Value = " +
                futureValue(presentValue, growthRate, years));
    }
}
```

## Output :

```
Future Value = 1331.0
```